

CO3 AT3 JAVA SCENARIO-BASED PROGRAMMING

ASSIGNMENT3

Scenario

An elevator display system simulates **three independent elevators** moving between floors. Each elevator moves one floor at a time with a small delay. The program displays the elevator's current floor and thread lifecycle information.

```
// Integrated Healthcare Pharmacy, Prescription and Stock Safety Platform
```

```
// Java Scenario-Based Multithreading Program
```

```
// Scenario: Three-Elevator Display Application
```

```
class ElevatorOne extends Thread {

    private int currentFloor;
    private int targetFloor;

    public ElevatorOne(String name, int startFloor, int targetFloor) {
        super(name);
        this.currentFloor = startFloor;
        this.targetFloor = targetFloor;
    }

    @Override
    public void run() {

        System.out.println("\n" + getName() + " started.");
        System.out.println("Current Thread : " + Thread.currentThread());
        System.out.println("Thread State  : " + getState());
        System.out.println("Thread Priority: " + getPriority());

        try {
```

```
while (currentFloor != targetFloor) {

    if (currentFloor < targetFloor) {
        currentFloor++;
    } else {
        currentFloor--;
    }

    System.out.println(getName() +
        " is moving... Current Floor: " + currentFloor);

    Thread.sleep(700);
}

System.out.println(getName() +
    " reached destination floor " + targetFloor);

} catch (InterruptedException e) {

    System.out.println(getName() +
        " was interrupted.");

    Thread.currentThread().interrupt();
}

System.out.println(getName() + " finished.");
}
```

```
// Elevator implemented using Runnable
class ElevatorTwo implements Runnable {

    private int currentFloor;

    private int targetFloor;

    public ElevatorTwo(int startFloor, int targetFloor) {

        this.currentFloor = startFloor;

        this.targetFloor = targetFloor;

    }

    @Override
    public void run() {

        Thread currentThread = Thread.currentThread();

        System.out.println("\n" + currentThread.getName() + " started.");
        System.out.println("Current Thread : " + currentThread);
        System.out.println("Thread State : " + currentThread.getState());
        System.out.println("Thread Priority: " + currentThread.getPriority());

        try {

            while (currentFloor != targetFloor) {

                if (currentFloor < targetFloor) {

                    currentFloor++;

                }

            }

        }

    }

}
```

```

    } else {
        currentFloor--;
    }

    System.out.println(currentThread.getName() +
        " is moving... Current Floor: " + currentFloor);

    Thread.sleep(900);
}

System.out.println(currentThread.getName() +
    " reached destination floor " + targetFloor);

} catch (InterruptedException e) {

    System.out.println(currentThread.getName() +
        " was interrupted.");

    Thread.currentThread().interrupt();
}

System.out.println(currentThread.getName() + " finished.");
}
}

```

```

// Elevator implemented using Runnable
class ElevatorThree implements Runnable {

```

```
private int currentFloor;
```

```
private int targetFloor;
```

```
public ElevatorThree(int startFloor, int targetFloor) {
```

```
    this.currentFloor = startFloor;
```

```
    this.targetFloor = targetFloor;
```

```
}
```

```
@Override
```

```
public void run() {
```

```
    Thread currentThread = Thread.currentThread();
```

```
    System.out.println("\n" + currentThread.getName() + " started.");
```

```
    System.out.println("Current Thread : " + currentThread);
```

```
    System.out.println("Thread State : " + currentThread.getState());
```

```
    System.out.println("Thread Priority: " + currentThread.getPriority());
```

```
    try {
```

```
        while (currentFloor != targetFloor) {
```

```
            if (currentFloor < targetFloor) {
```

```
                currentFloor++;
```

```
            } else {
```

```
                currentFloor--;
```

```
            }
```

```
            System.out.println(currentThread.getName() +
```

```

        " is moving... Current Floor: " + currentFloor);

        Thread.sleep(500);
    }

    System.out.println(currentThread.getName() +
        " reached destination floor " + targetFloor);

} catch (InterruptedException e) {

    System.out.println(currentThread.getName() +
        " was interrupted.");

    Thread.currentThread().interrupt();
}

System.out.println(currentThread.getName() + " finished.");
}
}

// Main class
public class ElevatorDisplayApplication {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("    ELEVATOR DISPLAY APPLICATION");
        System.out.println("    MULTITHREADING JAVA PROGRAM");
    }
}

```

```
System.out.println("=====");
```

```
// Elevator 1 using Thread class
```

```
ElevatorOne elevator1 =
```

```
    new ElevatorOne("Elevator-1", 1, 8);
```

```
// Elevator 2 using Runnable interface
```

```
ElevatorTwo elevatorTask2 =
```

```
    new ElevatorTwo(10, 3);
```

```
Thread elevator2 =
```

```
    new Thread(elevatorTask2, "Elevator-2");
```

```
// Elevator 3 using Runnable interface
```

```
ElevatorThree elevatorTask3 =
```

```
    new ElevatorThree(5, 12);
```

```
Thread elevator3 =
```

```
    new Thread(elevatorTask3, "Elevator-3");
```

```
// Assign different priorities as scheduling hints
```

```
elevator1.setPriority(Thread.NORM_PRIORITY);
```

```
elevator2.setPriority(Thread.MAX_PRIORITY);
```

```
elevator3.setPriority(Thread.MIN_PRIORITY);
```

```
System.out.println("\n--- BEFORE STARTING THREADS ---");
```

```
System.out.println(elevator1.getName() +  
    " | State: " + elevator1.getState() +  
    " | Alive: " + elevator1.isAlive() +  
    " | Priority: " + elevator1.getPriority());
```

```
System.out.println(elevator2.getName() +  
    " | State: " + elevator2.getState() +  
    " | Alive: " + elevator2.isAlive() +  
    " | Priority: " + elevator2.getPriority());
```

```
System.out.println(elevator3.getName() +  
    " | State: " + elevator3.getState() +  
    " | Alive: " + elevator3.isAlive() +  
    " | Priority: " + elevator3.getPriority());
```

```
System.out.println("\n--- STARTING ELEVATORS ---");
```

```
// Correct use of start()
```

```
elevator1.start();
```

```
elevator2.start();
```

```
elevator3.start();
```

```
// Check thread status after starting
```

```
System.out.println("\n--- THREAD STATUS AFTER START() ---");
```

```
System.out.println(elevator1.getName() +  
    " | State: " + elevator1.getState() +
```



```
    " | Alive: " + elevator1.isAlive());
```

```
System.out.println(elevator2.getName() +
```

```
    " | State: " + elevator2.getState() +
```

```
    " | Alive: " + elevator2.isAlive());
```

```
System.out.println(elevator3.getName() +
```

```
    " | State: " + elevator3.getState() +
```

```
    " | Alive: " + elevator3.isAlive());
```

```
// Main thread also performs independent work
```

```
System.out.println("\nMain Thread: " +
```

```
    Thread.currentThread().getName());
```

```
System.out.println("Main Thread State: " +
```

```
    Thread.currentThread().getState());
```

```
try {
```

```
    Thread.sleep(1000);
```

```
} catch (InterruptedException e) {
```

```
    System.out.println("Main thread was interrupted.");
```

```
    Thread.currentThread().interrupt();
```

```
}
```

```

// join() ensures final report comes after all elevators finish
try {

    elevator1.join();
    elevator2.join();
    elevator3.join();

} catch (InterruptedException e) {

    System.out.println("Main thread was interrupted while waiting.");
    Thread.currentThread().interrupt();
}

// Final report
System.out.println("\n=====");
System.out.println("        FINAL ELEVATOR REPORT");
System.out.println("=====");

System.out.println(elevator1.getName() +
    " | State: " + elevator1.getState() +
    " | Alive: " + elevator1.isAlive());

System.out.println(elevator2.getName() +
    " | State: " + elevator2.getState() +
    " | Alive: " + elevator2.isAlive());

System.out.println(elevator3.getName() +
    " | State: " + elevator3.getState() +

```

```

        " | Alive: " + elevator3.isAlive());

    System.out.println("\nAll elevator threads have completed.");
    System.out.println("Main thread completed the final report.");

    System.out.println("=====");
}
}

```

What this program demonstrates

Requirement	Implementation
Three threads	Elevator-1, Elevator-2, Elevator-3
Thread class	ElevatorOne extends Thread
Runnable interface	ElevatorTwo, ElevatorThree
Meaningful names	"Elevator-1", "Elevator-2", "Elevator-3"
start()	Used to start all three elevators
sleep()	Used to simulate elevator movement
currentThread()	Displays the currently executing thread
isAlive()	Checks whether each thread is running
Priority	MIN, NORM, and MAX priorities assigned
Thread.State	Displays NEW, RUNNABLE, TIMED_WAITING, TERMINATED etc.
join()	Main waits for all elevators before final report
Exception handling	InterruptedException handled
Independent work	Each elevator has its own starting/target floor
No execution-order assumption	Threads are started independently

Important point for your viva

Thread priority does not guarantee execution order.

For example, Elevator-2 has MAX_PRIORITY, but this does **not** mean it must finish first. The operating system/JVM scheduler decides the actual execution order.

Expected output

The exact order will vary because the three elevators execute concurrently.

=====

ELEVATOR DISPLAY APPLICATION

MULTITHREADING JAVA PROGRAM

=====

--- BEFORE STARTING THREADS ---

Elevator-1 | State: NEW | Alive: false | Priority: 5

Elevator-2 | State: NEW | Alive: false | Priority: 10

Elevator-3 | State: NEW | Alive: false | Priority: 1

--- STARTING ELEVATORS ---

Elevator-1 started.

Elevator-2 started.

Elevator-3 started.

Elevator-1 is moving... Current Floor: 2

Elevator-2 is moving... Current Floor: 9

Elevator-3 is moving... Current Floor: 6

Elevator-3 is moving... Current Floor: 7

Elevator-1 is moving... Current Floor: 3

Elevator-2 is moving... Current Floor: 8

...

Elevator-1 reached destination floor 8

Elevator-2 reached destination floor 3

Elevator-3 reached destination floor 12

=====

FINAL ELEVATOR REPORT

=====

Elevator-1 | State: TERMINATED | Alive: false

Elevator-2 | State: TERMINATED | Alive: false

Elevator-3 | State: TERMINATED | Alive: false

All elevator threads have completed.

Main thread completed the final report.

=====

Note: Your given title *“Integrated Healthcare Pharmacy, Prescription and Stock Safety Platform”* doesn't match this particular programming scenario. For this question, the relevant application scenario is the **Elevator Display Application**; you can keep the healthcare title as your larger project title if your faculty requires it.

OUTPUT :

```
class ElevatorOne extends Thread {

    private int currentFloor;
    private int targetFloor;

    public ElevatorOne(String name, int startFloor, int targetFloor) {
        super(name);
        this.currentFloor = startFloor;
        this.targetFloor = targetFloor;
    }

    @Override
    public void run() {

        System.out.println("\n" + getName() + " started.");

        System.out.println("Current Thread: "
            + Thread.currentThread());

        System.out.println("Thread State: "
            + getState());

        System.out.println("Thread Priority: "
            + getPriority());

        try {

            while (currentFloor != targetFloor) {

                if (currentFloor < targetFloor) {
                    currentFloor++;
                } else {
                    currentFloor--;
                }

                System.out.println(getName()
                    + " -> Current Floor: "
                    + currentFloor);

                Thread.sleep(700);
            }
        }
    }
}
```

Ln 306, Col 2 7,751 characters Plain text 100% Windows (CRLF)

```
C:\Windows\System32\cmd.e X + v

C:\Users\Rajan\OneDrive\Desktop\javaprogram>java ElevatorDisplayApplication
=====
ELEVATOR DISPLAY SYSTEM
=====

--- BEFORE START ---
Elevator-1 | State: NEW | Alive: false
Elevator-2 | State: NEW | Alive: false
Elevator-3 | State: NEW | Alive: false

--- STARTING ELEVATORS ---

--- AFTER START ---

Elevator-2 started.
Current Thread: Thread[#26,Elevator-2,10,main]
Thread State: RUNNABLE
Elevator-1 | Alive: true
Elevator-2 | Alive: true
Elevator-3 | Alive: true

Elevator-1 started.
Current Thread: Thread[#25,Elevator-1,5,main]
Thread Priority: 10
Thread State: RUNNABLE
Thread Priority: 5
Elevator-2 -> Current Floor: 9

Elevator-3 started.
Current Thread: Thread[#27,Elevator-3,1,main]
Thread State: RUNNABLE
Thread Priority: 1
Elevator-3 -> Current Floor: 6
Elevator-1 -> Current Floor: 2
Elevator-3 -> Current Floor: 7
Elevator-1 -> Current Floor: 3
Elevator-2 -> Current Floor: 8
Elevator-3 -> Current Floor: 8
Elevator-1 -> Current Floor: 4
Elevator-3 -> Current Floor: 9
```

```
Thread State: RUNNABLE
Thread Priority: 5
Elevator-2 -> Current Floor: 9

Elevator-3 started.
Current Thread: Thread[#27,Elevator-3,1,main]
Thread State: RUNNABLE
Thread Priority: 1
Elevator-3 -> Current Floor: 6
Elevator-1 -> Current Floor: 2
Elevator-3 -> Current Floor: 7
Elevator-1 -> Current Floor: 3
Elevator-2 -> Current Floor: 8
Elevator-3 -> Current Floor: 8
Elevator-1 -> Current Floor: 4
Elevator-3 -> Current Floor: 9
Elevator-2 -> Current Floor: 7
Elevator-3 -> Current Floor: 10
Elevator-1 -> Current Floor: 5
Elevator-3 -> Current Floor: 11
Elevator-2 -> Current Floor: 6
Elevator-1 -> Current Floor: 6
Elevator-3 -> Current Floor: 12
Elevator-1 -> Current Floor: 7
Elevator-3 reached Floor 12
Elevator-2 -> Current Floor: 5
Elevator-1 -> Current Floor: 8
Elevator-2 -> Current Floor: 4
Elevator-1 reached Floor 8
Elevator-2 -> Current Floor: 3
Elevator-2 reached Floor 3

=====
                FINAL REPORT
=====
Elevator-1 | State: TERMINATED | Alive: false
Elevator-2 | State: TERMINATED | Alive: false
Elevator-3 | State: TERMINATED | Alive: false

All elevators completed.
```